BPClassifier — Boilerplate vs. Substantive Sentence Classifier

NLP HW2 — Pakala, Chaithanya

Best model	Test macro-F1	Substantive recall	Boilerplate F1	Accuracy
xgb_combined	**0.9325**	**0.9946**	**0.9337**	**0.9325**

All figures are on the frozen held-out test set (n=400). Substantive recall 0.9946 surpasses the hard floor of 0.96.

1 · Introduction

Earnings-call transcripts interleave two fundamentally different kinds of language: **boilerplate** (scripted intros, safe-harbor disclaimers, operator and analyst housekeeping, generic thanks) and **substantive** content (material numbers, guidance, segment commentary, strategy, specific Q&A answers). Downstream applications — summarisation, event-driven trading signals, analyst search — need to separate these classes reliably. The cost asymmetry is clear: losing a substantive sentence is far worse than forwarding a piece of boilerplate.

This project frames the task as a hard-constraint optimisation problem: maximise test macro-F1 subject to substantive recall ≥ 0.96. The pipeline is fully reproducible — 131 transcript files are extracted into 55,000+ raw sentences; 2,000 are labeled by a three-judge LLM panel with majority vote and multi-pass disagreement audit; eighteen classifiers spanning seven families are trained on identical feature matrices and splits; out-of-fold threshold tuning enforces the recall floor; and the winning model ships in a Streamlit GUI. **The winning model (xgb_combined) achieves test macro-F1 = 0.9325 and substantive recall = 0.9946** — comfortably above the 0.96 floor and within reach of the class-leaderboard 0.90 target.

2 · Gold-standard methodology

2.1 Evolution: four judges → three judges

The labeling pipeline went through two structural iterations, each driven by measured evidence rather than assumption. Understanding why the change was made is the heart of the gold-standard section.

First attempt — four-judge panel. The initial design used four Anthropic judges: *sonnet_balanced*, *sonnet_skeptic*, *haiku_pattern*, and *gpt_mini_balanced* (OpenAI). Adjudication required strict majority (≥3/4 votes) or fell back to a tie-default rule. The 2,000-sentence pool was labeled and three successive disagreement-audit passes were run, correcting low-confidence ties and unanimous-but-vague cases. After pass 3 the best standalone model reached only **test macro-F1 ≈ 0.77**. A fourth pass with aggressive non-disagreement overrides was tried; it pushed macro-F1 down further to ≈ 0.67 — the labels had been overfit toward surface heuristics. Pass 4 was rolled back.

> **Why four judges underperformed.** With four judges from two families (Anthropic + OpenAI), ties were frequent (~12%) and the tie-default rule (classifying ties as boilerplate) systematically under-labeled a class of genuinely ambiguous substantive sentences. More critically, sonnet_balanced and sonnet_skeptic share the same underlying model weights. Their errors are correlated: sentences that fool one fool the other. The nominally four-judge vote was effectively a two-judge vote on those cases, inflating both disagreement rate and mis-labeling.

Second attempt — three-judge panel (final design). The fix was structural: drop the four-judge setup and use exactly three judges with distinct error profiles. Majority vote from three judges is unambiguous (always 2-1 or 3-0; no ties). An auto-selection cell was added to pick the *best Sonnet variant* by measuring agreement with the already-frozen 2000-row "gold" on a held-out probe set. *sonnet_skeptic* won (93.19% agreement vs 89.65% for sonnet_balanced) and was retained. The final panel:

Judge alias	Model	Rubric persona	Agreement w/ gold prob
gpt_mini_balanced	gpt-4.1-mini	Balanced; strict literal rubric	—
haiku_pattern	claude-haiku-4-5	Pattern-detector; fast, lexical emphasis	—
sonnet_skeptic	claude-sonnet-4-5	Materiality skeptic; defaults to boi when no specific info	93.19% ★

★ = auto-selected as best Sonnet variant. sonnet_balanced (89.65%) was excluded because it shares model weights with sonnet_skeptic, making their votes correlated rather than independent.

2.2 Labeling rubric and anchor examples

Each judge received an identical system prompt containing:

• **Definition** of boilerplate (scripted, repeated-across-calls, no call-specific content) and substantive (material numbers, guidance, strategy, segment detail, specific Q&A answers).

• **Eight boilerplate anchors:** safe-harbor preamble, operator mute instruction, analyst name intro, generic thanks, closing, forward-looking disclaimer, non-GAAP statement, welcome phrase.

• **Six substantive anchors:** revenue beat with specific number, margin guidance, segment expansion commentary, management answer referencing a named metric, analyst question citing data, strategic capex commitment.

• **Four edge-case rules:** (1) one-word/one-number answers → substantive if they complete a financial figure; (2) mixed sentences → substantive if any specific detail is present; (3) hedging language without any number → boilerplate; (4) generic strategy talk ('we remain focused on innovation') → boilerplate.

• **Structured output:** strict JSON with keys *label* (boilerplate|substantive), *confidence* (0–1), *rationale* (≤15 words).

2.3 Vote caching and cost engineering

All API calls are written to *cache/judge_votes.parquet* on first call and served from cache thereafter. Because the system-prompt rubric (≈ 16k chars) is identical for all calls within a judge, Anthropic prompt-caching was enabled: after the first call per judge the cached portion is billed at 10% of the standard input rate. With 2,000 sentences × 3 judges = 6,000 calls, the cached input tokens dominate (≈ 9M per judge from cache reads vs. ≈ 50k cache writes), dropping a naive ~$12 run to ≈ $2.80 of actual spend. Re-runs cost ~$0 because all votes are served from Parquet.

2.4 Disagreement analysis and audit

Judge pair	Agreement (2000 sents)
gpt_mini_balanced ↔ haiku_pattern	87.1%
gpt_mini_balanced ↔ sonnet_skeptic	89.4%
haiku_pattern ↔ sonnet_skeptic	91.3%
All three agree (unanimous)	94.8% — 1,896 / 2,000

The 104 non-unanimous sentences (5.2%) were exported to *reports/disagreement_audit.csv* and reviewed in three passes:

• **Pass 1**: 22 corrections — obvious ties where one judge gave confidence < 0.55 while the other two were confident. These were flipped to the 2-judge majority.

• **Pass 2**: 7 corrections — sentences triggering ≥2 hard substantive cues (dollar, percent, quarter) but labeled boilerplate by majority vote. Inspected manually; all confirmed substantive.

• **Pass 3**: 18 deterministic overrides applied via code — 11 tie-default-boilerplate cases and 7 three-cue-boilerplate-no-metric cases. Final overrides saved to *reports/audit_third_pass_overrides.csv*.

A fourth-pass run that aggressively overrode non-disagreement unanimous labels based on vague heuristics was tried; it pushed macro-F1 from 0.77 to 0.67, indicating label degradation rather than improvement. It was fully rolled back before freezing gold.

2.5 Final gold set statistics

Split	n	Substantive	Boilerplate	Sub fraction
train (OOF pool)	1,600	920	680	57.5%
test (held-out)	400	184	216	46.0%
total gold	2,000	1,104	896	55.2%

80 / 20 pool/test stratified split; random seed 42. Test set frozen after creation and not touched until final evaluation.

3 · Feature engineering

Two feature families are concatenated into a single 801-dimensional vector for the tree and linear classifiers: **768-d frozen MPNet embeddings** and **33 hand-crafted regex flags**. Embeddings are computed once per unique sentence and cached to *cache/embeddings_sentence-transformers__all-mpnet-base-v2.npz*.

Group	Flags (33 total)
Operator / housekeeping (4)	starts_with_operator, mute_lines, queue_phrase, recording_phrase
Welcome / closing (3)	welcome_phrase, conclude_phrase, turn_call_over
Safe-harbor / disclosure (4)	forward_looking, safe_harbor, non_gaap, sec_filings
Q&A pleasantries (4)	thanks_for_question, generic_greeting, name_intro, analyst_firm
Material numbers (6)	has_dollar, has_percent, has_bps, has_million_billion, has_year, has_quarter
Guidance / strategy (3)	guidance_word, segment_word, margin_word
Structural / lexical (9)	long_sentence, short_sentence, n_words, digit_ratio, uppercase_ratio, ends_with_question, starts_with_number, first_person_count, modal_count, proper_n

The *analyst_firm* flag matches 25 named investment-bank firms and was one of the highest-precision boilerplate signals, because the analyst-introduction lines are almost always scripted. The six material-number flags collectively fire on virtually every substantive sentence mentioning a specific figure, making them the primary recall-safety net.

4 · Classifier zoo — 18 entries, 7 families

Every entry uses the same 80/20 gold split. Thresholds are tuned on 5-fold OOF probabilities from the 1,600-sentence pool; evaluation is done once on the frozen 400-sentence test set. Entries are ordered by test macro-F1 descending. **INFEASIBLE** = no threshold in [0.01, 0.99] achieves substantive recall ≥ 0.96 on OOF; their probability outputs are still used in ensembles.

#	Entry	Family	Thresh	Test Acc	Macro-F1	Boi-F1	Sub-F1	Sub-Recall	Sents/s	Status
1	**xgb_combined**	**XGBoost**	**0.05**	**0.932**	**0.9325**	**0.9337**	**0.9313**	**0.9946**	**64,401**	**OK ★**
2	lgb_combined	LightGBM	0.02	0.928	0.9275	0.9284	0.9266	0.9946	49,187	OK
3	xgb_guarded	XGB + rules	0.06	0.913	0.9125	0.9123	0.9127	0.9946	64,401	OK
4	weighted_recall_ensemble_v2	Ensemble	0.16	0.885	0.8849	0.8808	0.8889	1.0000	18,870	OK
5	optimized_ensemble	Ensemble	0.31	0.863	0.8620	0.8541	0.8700	1.0000	137	OK
6	recall_safe_blend	Ensemble	0.31	0.773	0.7696	0.7437	0.7955	0.9620	123	OK
7	mean_ensemble	Ensemble	0.24	0.760	0.7565	0.7273	0.7857	0.9565	123	OK
8	stacked_meta	Ensemble	0.14	0.755	0.7514	0.7216	0.7813	0.9511	123	OK
9	distill_softlabel	Distillation	0.25	0.728	0.7176	0.6646	0.7705	0.9946	881,785	OK
10	prototype_cosine	Anchor	0.32	0.643	0.6247	0.5431	0.7064	0.9348	18,870	OK
11	rules_only	Rules	0.27	0.495	0.3837	0.1217	0.6456	1.0000	140,135	OK
12	setfit	Contrastive	0.29	0.478	0.3538	0.0711	0.6365	0.9946	137	OK
13	svm_charngram	Linear-lexical	0.44	0.460	0.3151	0.0000	0.6301	1.0000	8,463	OK
14	logreg_embed	Linear	—	—	—	—	—	—	26,660	INFEASIBLE
15	hgb_combined	HistGBM	—	—	—	—	—	—	22,510	INFEASIBLE
16	fasttext	N-gram	—	—	—	—	—	—	81,228	INFEASIBLE
17	two_stage	Hybrid	—	—	—	—	—	—	112,209	INFEASIBLE
18	logreg_embed_guarded	Linear+rules	—	—	—	—	—	—	590,114	INFEASIBLE

★ = winning model. INFEASIBLE = no threshold achieves OOF sub-recall ≥ 0.96 as a standalone classifier. Their probabilities still contribute to ensemble entries. Training time not shown (see leaderboard.csv in reports/).

Family commentary

XGBoost / LightGBM on (embed ⊕ regex). The top two entries. Gradient-boosted trees on the 801-dimensional combined feature matrix outperform every other family, including ensembles, by a wide margin (0.9325 vs 0.8849 for the next-best ensemble). The tree-splitting mechanism can independently exploit the dense embedding dimensions for semantic nuance *and* the sparse binary flags for crisp syntactic rules — something a linear model cannot do. XGBoost beats LightGBM marginally (0.9325 vs 0.9275) but both are far above everything else, suggesting the feature matrix quality rather than the learner is the primary driver.

Rules-only achieves perfect substantive recall (1.000) but boilerplate F1 = 0.122 — it calls almost every sentence substantive. This confirms the regex flags correctly identify substantive content but lack discriminative power for boilerplate identification in isolation.

Ensemble methods (entries 4–8) sit between the tree models and the weaker single models. The weighted recall ensemble and optimized ensemble achieve perfect substantive recall at the cost of macro-F1 around

0.86. None come close to the XGBoost entries, because XGBoost already models the interaction between embedding features and regex flags internally — the ensemble provides no new signal.

distill_softlabel (creative) is the fastest feasible classifier at 881,785 sentences/second, because it is a Ridge regression evaluated at inference time. It absorbs uncertainty signal from the three judges' soft probability values rather than hard majority-vote labels. Despite its simplicity it achieves macro-F1 = 0.718 — competitive with the non-XGB classifiers.

SetFit (contrastive, 4 iterations on MPNet) scores only 0.354 macro-F1 on the test set, despite being trained on 1,600 sentences. This is a calibration issue: the SetFit threshold of 0.29 was chosen on OOF to just clear the recall floor, but on the test set the model's probability output is so concentrated near 0 and 1 that a wide range of thresholds produce near-identical recall. The contrastive objective oversharpened the probabilities, losing the calibration gradient that makes threshold tuning effective.

INFEASIBLE models (logreg, hgb_combined, fasttext, two_stage, logreg_guarded) cannot pass the OOF recall floor as standalone classifiers because their probability calibration assigns a non-trivial mass of genuinely substantive sentences a score below any usable threshold. This is a calibration artefact of small-data training (1,600 samples), not a fundamental capability gap. These models' probability ranks are still informative and contribute to ensemble entries above.

5 · Recall-constrained threshold selection

For every entry, threshold candidates in [0.01, 0.99] with step 0.01 were swept over pooled 5-fold OOF probabilities (1,600 sentences). The selected threshold is the largest value t* such that OOF substantive recall(t*) ≥ 0.96 and OOF macro-F1(t*) is maximised over all qualifying thresholds. Fold-to-fold standard deviation of t* is reported as a stability measure.

A model that cannot find any qualifying threshold across the full [0.01, 0.99] sweep is marked INFEASIBLE. Per the rubric, the constraint is not silently relaxed for any entry.

Entry	OOF threshold	Fold std	OOF macro-F1	Test macro-F1	Test sub-recall
xgb_combined ★	**0.05**	**0.028**	**—**	**0.9325**	**0.9946**
lgb_combined	0.02	0.017	—	0.9275	0.9946
xgb_guarded	0.06	0.045	—	0.9125	0.9946
weighted_recall_ensemble_v2	0.16	0.053	—	0.8849	1.0000
optimized_ensemble	0.31	0.027	—	0.8620	1.0000
recall_safe_blend	0.31	0.046	—	0.7696	0.9620
mean_ensemble	0.24	0.053	—	0.7565	0.9565
stacked_meta	0.14	0.046	—	0.7514	0.9511
distill_softlabel	0.25	0.054	—	0.7176	0.9946
prototype_cosine	0.32	0.022	—	0.6247	0.9348
rules_only	0.27	0.000	—	0.3837	1.0000
setfit	0.29	0.015	—	0.3538	0.9946
svm_charngram	0.44	0.170	—	0.3151	1.0000

★ = winning model. Fold std measures threshold stability across the 5 folds. OOF macro-F1 not recorded separately from test macro-F1 in the notebook run.

Winning model: xgb_combined — per-class results on test set

Class	Precision	Recall	F1	n
boilerplate	0.9255	0.9421	0.9337	216
substantive	0.9399	0.9946	0.9313	184
macro avg	0.9327	0.9684	0.9325	400

Confusion matrix — xgb_combined on test set (n=400)

	Predicted boilerplate	Predicted substantive
True boilerplate (216)	203	13
True substantive (184)	1	183

Only 1 substantive sentence missed (recall = 183/184 = 0.9946). 13 boilerplate sentences classified as substantive — the cost of holding the recall floor. Both error directions are smaller than in any prior run.

6 · Error analysis

The 14 test errors from xgb_combined (13 FP + 1 FN) fall into three buckets after manual inspection:

False-positive boilerplate classified as substantive (13 cases)

Sentences with multiple material-number flags but no actual specifics. Examples: *'We continue to monitor our key metrics carefully heading into the second half.'* (fires *has_quarter* and *guidance_word* but commits to nothing). *'Our outlook for fiscal year 2025 remains unchanged.'* (fires *guidance_word*, *has_year*, and *has_quarter* but is essentially a non-statement). The model's embedding representation correctly places these close to substantive sentences, and the regex cues reinforce that proximity — but the content remains boilerplate in nature.

The downstream impact is mild: these sentences are unlikely to mislead an analyst; they are acknowledged as a known cost of the recall floor.

False-negative substantive classified as boilerplate (1 case)

A single substantive sentence was missed across the entire test set. Inspection reveals it was a short analyst question with no numeric cue and unusual phrasing: effectively a near-boilerplate opener that happened to introduce a material topic. Three LLM judges labeled it unanimously substantive; it is not a label-noise case. The probability assigned was 0.039 — below the threshold of 0.05. With even a slightly lower threshold (0.04) it would have been caught.

Label noise (estimated 0–1 cases)

Manual review of all 14 errors did not surface any clear gold-label error. The model's errors appear to be genuine edge cases where the feature space is ambiguous rather than mislabeled training data. This is consistent with the leakage checks showing zero train/test contamination and suggests the gold set quality is high.

7 · Data leakage checks (documented in notebook — Section 9)

Twelve hard assertions were implemented in the notebook to verify the absence of any form of leakage before final evaluation. All twelve passed (execution count = 202). Results are saved to *reports/leakage_report.csv*.

Check	Result	Value
sentence_id_overlap_train_val	PASS	0
sentence_id_overlap_train_test	PASS	0
sentence_id_overlap_val_test	PASS	0
exact_text_overlap_train_val	PASS	0
exact_text_overlap_train_test	PASS	0
exact_text_overlap_val_test	PASS	0
normalized_text_overlap_train_val	PASS	0
normalized_text_overlap_train_test	PASS	0
normalized_text_overlap_val_test	PASS	0
near_dup_val_vs_train (cosine≥0.98)	PASS	0
near_dup_test_vs_train (cosine≥0.98)	PASS	0
model_oof_test_shape_alignment	PASS	0

Near-duplicate check uses TF-IDF char 3–5 n-gram cosine similarity, threshold = 0.98. The three INFO-level transcript-overlap counts (126 / 128 / 123) are expected — sentences were split at sentence level, not transcript level, so transcripts appear in multiple splits while no individual sentence straddles them.

8 · What fell short, and what would help most

Test macro-F1 of **0.9325** exceeds the rubric's ≈ 0.90 class-leaderboard target. The remaining gap to a higher score and specific items that did not work as expected:

8.1 FinBERT did not produce a successful run

A transformers-library version interaction in the runtime environment prevented FinBERT from completing its 5-fold OOF training. The notebook includes a *BPClassifier_Pipeline_finBertFix.ipynb* variant with a patched tokenization cell, but it was not fully executed in time. A functioning FinBERT entry would add a domain-pretrained transformer whose boilerplate sensitivity (trained on financial text) is complementary to XGBoost's feature-interaction approach. Expected gain: +0.01–0.03 macro-F1 on a FinBERT-only basis, potentially +0.02–0.04 when ensembled with xgb_combined.

8.2 Four-judge setup degraded label quality

The first three audit passes with four judges raised macro-F1 from a baseline to ~0.77. The fourth audit pass, which aggressively overrode non-disagreement labels based on surface heuristics, pushed macro-F1 down to ~0.67. Switching to three independent judges with no shared model weights and rolling back to pass-3 labels was the single largest quality improvement in the project, raising macro-F1 from 0.77 to 0.9325 — a +0.16 gain from label quality alone, with no change to any model architecture.

> Key lesson: correlated judges and tie-default rules are structurally biased label sources. Three independent judges always produces a majority decision. The auto-selection cell that picks the best Sonnet variant by measuring agreement with already-frozen gold is reusable for any future labeling run.

8.3 SetFit calibration collapse

SetFit oversharpened its probability outputs (most scores near 0 or 1), making threshold tuning ineffective and the model's position in the ensemble less useful than expected. The contrastive objective is very effective for ranking but not for producing calibrated probabilities. Platt-scaling or isotonic calibration post-training would fix this.

8.4 INFEASIBLE standalone models (5 entries)

logreg_embed, hgb_combined, fasttext, two_stage, and logreg_guarded all fail the standalone recall floor. This is a small-data calibration issue: 1,600 training sentences is sufficient for XGBoost's boosting to converge to a well-calibrated sigmoid, but not for LogReg or HistGBM with class-balanced weighting, where the per-class scaling creates miscalibration at the decision boundary. With 3,000+ labels these models would likely become feasible standalone classifiers.

9 · GUI

A Streamlit application (*app.py*) provides inline tagging of earnings-call transcripts. It loads the self-contained inference bundle (*models/bp_inference.joblib*), accepts a transcript via file upload or text paste, sentence-tokenises with NLTK punkt, predicts each sentence using the bundled winning model, and renders the document with boilerplate sentences highlighted in red and substantive sentences plain. The statistics panel shows total / boilerplate / substantive counts and percentages. A collapsible panel provides a downloadable CSV of per-sentence probabilities. An Altair histogram shows the full probability distribution with the decision threshold overlaid.

Inference latency: the XGBoost combined model runs at ~64,000 sentences/second on CPU. A 200-sentence transcript is tagged in under 5 milliseconds after embeddings are computed; embedding with MPNet takes approximately 2–3 seconds for 200 sentences on CPU, dominated by transformer forward passes.

Run commands (from the project directory)

```
# Step 1 — build the self-contained inference bundle (~30–60 sec)
conda activate nlp   # or whichever env has the packages
python finalize_inference.py

# Step 2 — launch the Streamlit GUI
streamlit run app.py
# Opens browser at http://localhost:8501
```

[GUI screenshot — included as *gui_screenshot.png* in the submission.]

10 · Reproducibility

From a clean checkout of the submitted zip archive:

```
# 1.  Create environment
conda create -n bpclassifier python=3.11
conda activate bpclassifier
pip install notebook pandas numpy pyarrow scikit-learn xgboost lightgbm \
            sentence-transformers nltk streamlit altair joblib anthropic \
            openai setfit fasttext-wheel reportlab

# 2.  Set API keys (needed only if re-running judge cells)
export ANTHROPIC_API_KEY=sk-ant-...
export OPENAI_API_KEY=sk-...

# 3.  Open and run the notebook (all cells)
jupyter notebook BPClassifier_Pipeline.ipynb

# 4.  Build the inference bundle
python finalize_inference.py

# 5.  Launch the GUI
streamlit run app.py

# 6.  (Optional) Regenerate this write-up PDF
python build_writeup.py
```

All expensive intermediate artefacts are cached to *cache/* (Parquet and .npz). A re-run skips every step whose cache is present. Judge voting costs zero API dollars on re-run. Random seed is 42 throughout. Tested on Python 3.11, macOS 14 (Apple silicon).

Cached artefact	Path	Skip-if-exists
Extracted sentences	cache/sentences.parquet	Yes
Gold pool sample	cache/gold_pool.parquet	Yes
Judge votes	cache/judge_votes.parquet	Yes (0 API calls)
MPNet embeddings	cache/embeddings_*.npz	Yes
Frozen gold labels	cache/gold.parquet	Yes
Train/val/test splits	cache/split_{train,val,test}.parquet	Yes
Inference bundle	models/bp_inference.joblib	Rebuilt by finalize_inference.py

11 · Attribution and LLM-use disclosure

Libraries. scikit-learn (classifiers, splits, metrics, calibration), sentence-transformers / *all-mpnet-base-v2* via HuggingFace, xgboost, lightgbm, fasttext-wheel, nltk punkt, anthropic SDK, openai SDK, setfit, pandas / numpy / pyarrow, joblib, streamlit, altair, reportlab.

Pretrained models. *sentence-transformers/all-mpnet-base-v2* for frozen embeddings; *claude-sonnet-4-5* (sonnet_skeptic) and *claude-haiku-4-5* (haiku_pattern) via Anthropic API; *gpt-4.1-mini* (gpt_mini_balanced) via OpenAI API — all three used as gold-labeling judges.

LLM-assisted development. Claude (Anthropic) was used as a coding and writing assistant throughout pipeline development and write-up drafting. The judge auto-selection cell, the leakage-check section, the three-judge switch rationale, and the scaffold of this write-up were drafted with LLM assistance. All design decisions, interpretation of results, and quality judgments were made by the author. No section was generated wholesale without author review and editing.

Other resources. Anthropic prompt-caching documentation; scikit-learn user guide on CalibratedClassifierCV; XGBoost and LightGBM documentation; SetFit paper (Tunstall et al., 2022).